Universidad ORT Uruguay
Facultad de Ingeniería
Escuela de Tecnología

OBLIGATORIO PROGRAMACIÓN 2



Sebastian Pi – 271178

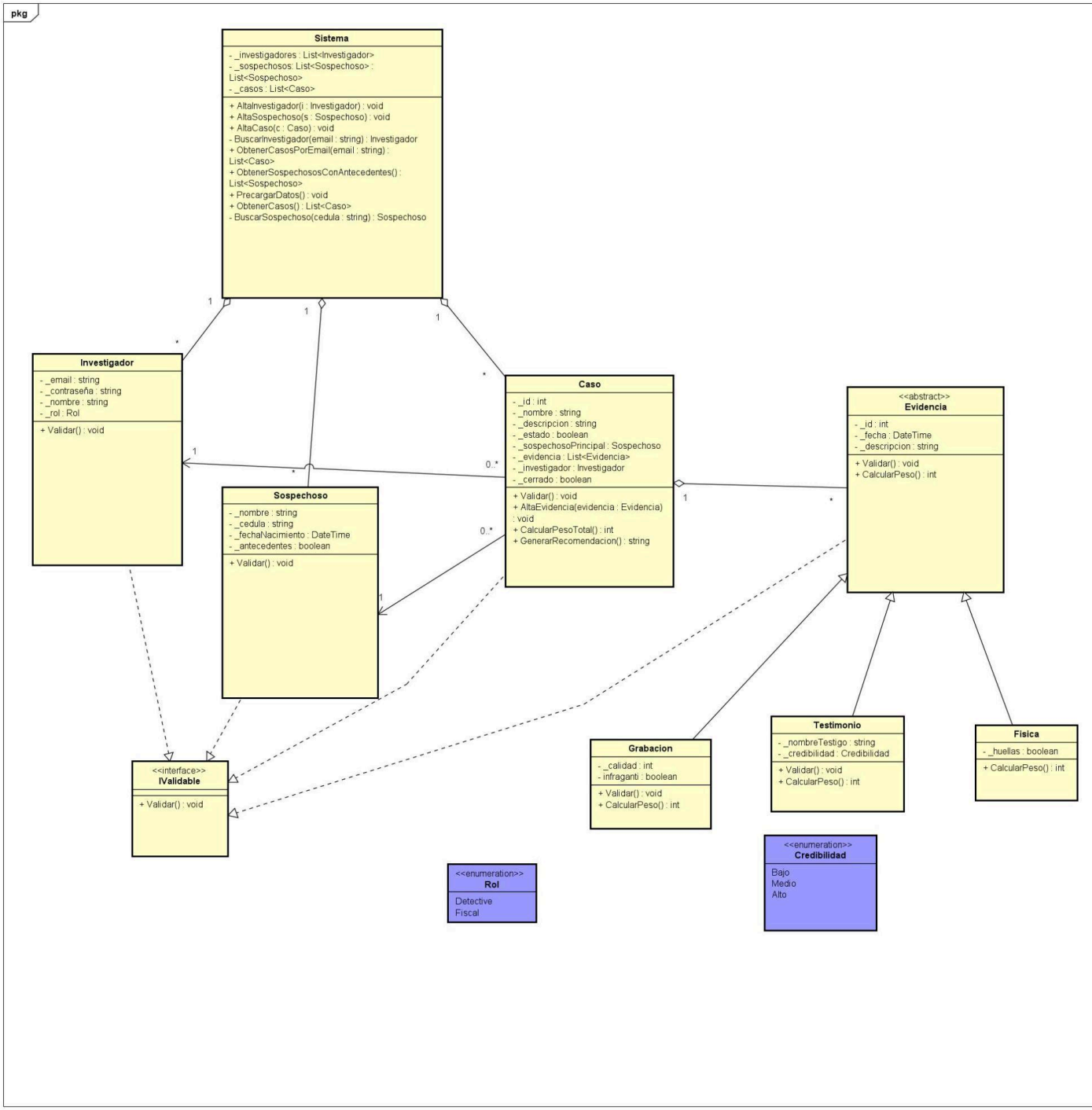
Grupo: SebaPi

Docente: Sergio Palay

Analista programador / Analista en tecnologías de la información.

15/10/2026

UML:


pkg
Sistema
- _investigadores : List<Investigador>
- _sospechosos: List<Sospechoso> : List<Sospechoso>
- _casos : List<Caso>
+ AltaInvestigador(i : Investigador) : void
+ AltaSospechoso(s : Sospechoso) : void
+ AltaCaso(c : Caso) : void
- BuscarInvestigador(email : string) : Investigador
+ ObtenerCasosPorEmail(email : string) : List<Caso>
+ ObtenerSospechososConAntecedentes() : List<Sospechoso>
+ PrecargarDatos() : void
+ ObtenerCasos() : List<Caso>
- BuscarSospechoso(cedula : string) : Sospechoso
Investigador
- _email : string
- _contraseña : string
- _nombre : string
- _rol : Rol
+ Validar() : void
Caso
- _id : int
- _nombre : string
- _descripcion : string
- _estado : boolean
- _sospechosoPrincipal : Sospechoso
- _evidencia : List<Evidencia>
- _investigador : Investigador
- _cerrado : boolean
+ Validar() : void
+ AltaEvidencia(evidencia : Evidencia) : void
+ CalcularPesoTotal() : int
+ GenerarRecomendacion() : string
<<abstract>>
Evidencia
- _id : int
- _fecha : DateTime
- _descripcion : string
+ Validar() : void
+ CalcularPeso() : int
Sospechoso
- _nombre : string
- _cedula : string
- _fechaNacimiento : DateTime
- _antecedentes : boolean
+ Validar() : void
<<interface>>
IValidable
+ Validar() : void
Grabacion
- _calidad : int
- infraganti : boolean
+ Validar() : void
+ CalcularPeso() : int
Testimonio
- _nombreTestigo : string
- _credibilidad : Credibilidad
+ Validar() : void
+ CalcularPeso() : int
Fisica
- _huellas : boolean
+ CalcularPeso() : int
<<enumeration>>
Rol
Detective
Fiscal
<<enumeration>>
Credibilidad
Bajo
Medio
Alto
1
*
0..*

Tablas de investigadores, casos, sospechosos y evidencia:

Investigadores

Email	Contraseña	Nombre	Rol
martin.silva@fiscalia.uy	Martin123	Martín Silva	DETECTIVE
lucia.fernandez@fiscalia.uy	Lucia456	Lucía Fernández	DETECTIVE
diego.rodriguez@fiscalia.uy	Diego789	Diego Rodríguez	DETECTIVE
sofia.pereira@fiscalia.uy	Sofia321	Sofía Pereira	DETECTIVE
nicolas.gomez@fiscalia.uy	Nico654	Nicolás Gómez	DETECTIVE
valentina.lopez@fiscalia.uy	Vale123	Valentina López	FISCAL
andres.martinez@fiscalia.uy	Andres456	Andrés Martínez	FISCAL

camila.suarez@fiscalia.uy	Cami789	Camila Suárez	FISCAL
federico.costa@fiscalia.uy	Fede321	Federico Costa	FISCAL
paula.ramos@fiscalia.uy	Paula654	Paula Ramos	FISCAL

Tabla de sospechosos:

Nombre	Cédula	Fecha nacimiento	Antecedentes
Carlos Méndez	4.521.630-1	14/05/1985	Sí
Bruno Acosta	5.184.920-3	22/09/1992	No
Mateo Cabrera	4.792.315-6	03/02/1988	Sí
Rodrigo Núñez	5.321.874-2	17/11/1995	No
Gonzalo Viera	4.638.291-5	28/07/1983	Sí
Joaquín Castro	5.472.103-8	11/01/1998	No
Sebastián Torres	4.819.562-4	06/06/1990	Sí
Agustín Morales	5.203.741-9	19/03/1994	No
Santiago Reyes	4.376.918-7	25/12/1981	Sí

Emiliano Sosa	5.619.284-1	08/08/1997	No
Leonardo Díaz	4.927.350-6	30/04/1987	Sí
Franco Olivera	5.738.421-3	15/10/2000	No
Mauricio Lima	4.583.127-9	09/01/1984	Sí
Facundo Pereira	5.346.892-5	21/06/1996	No
Pablo Ríos	4.715.263-8	12/03/1989	Sí

Tabla de casos:

#	Nombre	Descripción	Cerrado	Sospechoso principal	Investigador
1	Robo en Pocitos	Robo ocurrido durante la madrugada en una vivienda.	No	Carlos Méndez	Martín Silva
2	Hurto del Centro	Hurto de equipos electrónicos en un comercio.	No	Bruno Acosta	Lucía Fernández
3	Fraude Bancario	Investigación por	No	Mateo Cabrera	Diego Rodrígue

		movimientos bancarios fraudulentos.			z
4	Robo del Puerto	Robo de mercadería almacenada en la zona portuaria.	Sí	Rodrigo Núñez	Sofía Pereira
5	Caso Malvín	Investigación por ingreso ilegal a una vivienda.	No	Gonzalo Viera	Nicolás Gómez
6	Operación Parque Rodó	Investigación por robo de un vehículo.	No	Joaquín Castro	Martín Silva
7	Incidente Cordón	Robo ocurrido en un local comercial.	Sí	Sebastián Torres	Lucía Fernández
8	Caso Tres Cruces	Hurto de pertenencias ocurrido en la terminal.	No	Agustín Morales	Diego Rodríguez

9	Operación Carrasco	Robo cometido en una residencia.	No	Santiago Reyes	Sofía Pereira
10	Caso Aguada	Robo de mercadería de un depósito.	No	Leonardo Díaz	Nicolás Gómez

Tabla de Evidencias:

Caso	Tipo	Fecha	Descripción	Datos específicos
1	Física	10/09/2026	Destornillador encontrado junto a una ventana.	Huellas: Sí
1	Grabación	10/09/2026	Cámara exterior de la vivienda.	Calidad: 4 / Infraganti: Sí
1	Testimonio	11/09/2026	Vecino observó salir a una persona.	Roberto Pérez / ALTO
1	Física	11/09/2026	Guante encontrado en el jardín.	Huellas: No
2	Grabación	02/09/2026	Cámara ubicada frente al comercio.	Calidad: 3 / Infraganti: Sí

2	Física	02/09/2026	Mochila encontrada cerca del comercio.	Huellas: No
2	Testimonio	03/09/2026	Empleado vio al sospechoso.	Ana Gómez / MEDIO
2	Grabación	03/09/2026	Cámara de un comercio vecino.	Calidad: 2 / Infraganti: No
3	Física	20/08/2026	Documentación bancaria incautada.	Huellas: Sí
3	Testimonio	21/08/2026	Declaración de empleado bancario.	Marcos Suárez / ALTO
3	Grabación	21/08/2026	Cámara de cajero automático.	Calidad: 5 / Infraganti: Sí
3	Testimonio	22/08/2026	Declaración de cliente del banco.	Laura Díaz / BAJO
4	Grabación	14/07/2026	Cámara de seguridad del depósito.	Calidad: 4 / Infraganti: Sí
4	Física	14/07/2026	Herramienta utilizada sobre un contenedor.	Huellas: Sí

4	Testimonio	15/07/2026	Guardia observó movimientos sospechosos.	José Martínez / ALTO
4	Física	15/07/2026	Prenda encontrada en el depósito.	Huellas: No
5	Física	05/06/2026	Vaso encontrado dentro de la vivienda.	Huellas: Sí
5	Grabación	05/06/2026	Cámara ubicada en la entrada.	Calidad: 2 / Infraganti: No
5	Testimonio	06/06/2026	Vecina escuchó ruidos de madrugada.	María Rodríguez / MEDIO

5	Grabación	06/06/2026	Cámara de una vivienda cercana.	Calidad: 4 / Infraganti: Sí
6	Física	01/05/2026	Herramienta encontrada junto al vehículo.	Huellas: No
6	Grabación	01/05/2026	Cámara del	Calidad: 5 /

			estacionamiento.	Infraganti: Sí
6	Testimonio	02/05/2026	Testigo vio al sospechoso junto al auto.	Pedro López / ALTO
6	Física	02/05/2026	Botella encontrada dentro del vehículo.	Huellas: Sí
7	Testimonio	10/04/2026	Empleado observó ingresar al sospechoso.	Lucía Alonso / ALTO
7	Grabación	10/04/2026	Video de seguridad del local.	Calidad: 3 / Infraganti: Sí
7	Física	11/04/2026	Bolsa encontrada dentro del comercio.	Huellas: Sí
7	Testimonio	11/04/2026	Persona frente al comercio declaró.	Andrés Costa / BAJO
8	Grabación	12/03/2026	Cámara de la terminal.	Calidad: 4 / Infraganti: Sí
8	Testimonio	12/03/2026	Pasajero presenció el hurto.	Carolina Ramos / MEDIO

8	Física	13/03/2026	Objeto abandonado próximo al lugar.	Huellas: No
8	Grabación	13/03/2026	Cámara de otro sector de la terminal.	Calidad: 2 / Infraganti: No
9	Física	20/02/2026	Herramienta encontrada en el patio.	Huellas: Sí
9	Grabación	20/02/2026	Cámara de seguridad de la residencia.	Calidad: 5 / Infraganti: Sí
9	Testimonio	21/02/2026	Jardinero vio al sospechoso cerca.	Fernando Silva / ALTO
9	Física	21/02/2026	Gorra encontrada junto a una puerta.	Huellas: No
10	Grabación	15/01/2026	Grabación del interior del depósito.	Calidad: 3 / Infraganti: Sí
10	Física	15/01/2026	Caja manipulada durante el robo.	Huellas: Sí

10	Testimonio	16/01/2026	Trabajador vio al sospechoso.	Natalia Fernández / MEDIO
10	Grabación	16/01/2026	Cámara exterior del depósito.	Calidad: 1 / Infraganti: No

Prompt de chat gpt :

Me podrias dar una precarga de datos tomando en cuenta esto? Estos son los atributos que tienen cada clase

Sospechoso: "sospechosos que se registran en el sistema de Fiscalía se conoce el nombre, la cédula (única), la fecha de nacimiento y si tiene antecedentes o no."

Casos: "los casos se conoce un id numérico auto incremental, el nombre del caso, su descripción, si ya está cerrado o no, el sospechoso principal, el investigador que debe tener rol “DETECTIVE” y la lista de todas las evidencias recabadas para dicho caso."

Investigador: "investigadores de los cuales se conoce un email que debe ser único entre los investigadores, una contraseña, su nombre y el rol que cumple dicho investigador. El rol puede ser DETECTIVE o FISCAL."
Evidencia: "De las evidencias se registra un id numérico auto incremental, la fecha cuando se recolectó la
evidencia (no puede ser superior al día de registro de esta) y una descripción.
Una evidencia puede ser Física, Grabación o Testimonio. De las evidencias Físicas, además de lo

que sabemos de todas las evidencias, se debe registrar si la misma tiene huellas digitales del
sospechoso o no. De las Grabaciones también se debe saber su calidad (numero entero del 1 al 5) y
si la grabación captura al sospechoso infraganti delito o no. Para terminar, de los Testimonios,
también debemos tener en cuenta el nombre del testigo y su índice de credibilidad que puede ser
BAJO, MEDIO o ALTO.
Cada caso puede tener múltiples evidencias de varios tipos, por ejemplo, 2 testimonios, una prueba
física y 3 grabaciones distintas del hecho."

- Investigadores: 10 (5 con rol Fiscal, y 5 con rol DETECTIVE)
- Sospechosos: 15
- Casos: 10
- Evidencias: Cada caso debe tener al menos 4 evidencias asociadas de diferentes tipos.

Respuesta de ChatGPT:

Sí. Tomando únicamente esos requisitos, una precarga útil para probar distintos escenarios podría ser esta.

Investigadores — 10

Email	Contraseña	Nombre	Rol
martin.silva@fiscalia.uy	Martin123	Martín Silva	DETECTIVE

lucia.fernandez@fiscalia.uy	Lucia456	Lucía Fernández	DETECTIVE
diego.rodriguez@fiscalia.uy	Diego789	Diego Rodríguez	DETECTIVE
sofia.pereira@fiscalia.uy	Sofia321	Sofía Pereira	DETECTIVE
nicolas.gomez@fiscalia.uy	Nico654	Nicolás Gómez	DETECTIVE
valentina.lopez@fiscalia.uy	Vale123	Valentina López	FISCAL
andres.martinez@fiscalia.uy	Andres456	Andrés Martínez	FISCAL
camila.suarez@fiscalia.uy	Cami789	Camila Suárez	FISCAL
federico.costa@fiscalia.uy	Fede321	Federico Costa	FISCAL
paula.ramos@fiscalia.uy	Paula654	Paula Ramos	FISCAL

Así tenés exactamente **5 DETECTIVE y 5 FISCAL**, con emails diferentes.

Sospechosos — 15

Nombre	Cédula	Fecha nacimiento	Antecedentes
Carlos Méndez	4.521.630-1	14/05/1985	Sí
Bruno Acosta	5.184.920-3	22/09/1992	No
Mateo Cabrera	4.792.315-6	03/02/1988	Sí
Rodrigo Núñez	5.321.874-2	17/11/1995	No
Gonzalo Viera	4.638.291-5	28/07/1983	Sí
Joaquín Castro	5.472.103-8	11/01/1998	No
Sebastián Torres	4.819.562-4	06/06/1990	Sí
Agustín Morales	5.203.741-9	19/03/1994	No
Santiago Reyes	4.376.918-7	25/12/1981	Sí
Emiliano Sosa	5.619.284-1	08/08/1997	No
Leonardo Díaz	4.927.350-6	30/04/1987	Sí
Franco Olivera	5.738.421-3	15/10/2000	No

Mauricio Lima	4.583.127-9	09/01/1984	Sí
Facundo Pereira	5.346.892-5	21/06/1996	No
Pablo Ríos	4.715.263-8	12/03/1989	Sí

Las 15 cédulas son diferentes para cumplir con la unicidad.

Casos — 10

El **ID es autoincremental**, por lo que los números del 1 al 10 sirven solamente para identificar cada caso en esta propuesta.

#	Nombre	Descripción	Cerrado	Sospechoso principal	Investigador
1	Robo en Pocitos	Robo ocurrido durante la madrugada en una vivienda.	No	Carlos Méndez	Martín Silva
2	Hurto del Centro	Hurto de equipos electrónicos en un comercio.	No	Bruno Acosta	Lucía Fernández
3	Fraude Bancario	Investigación por movimientos	No	Mateo Cabrera	Diego Rodríguez

		bancarios fraudulentos.			
4	Robo del Puerto	Robo de mercadería almacenada en la zona portuaria.	Sí	Rodrigo Núñez	Sofía Pereira
5	Caso Malvín	Investigación por ingreso ilegal a una vivienda.	No	Gonzalo Viera	Nicolás Gómez
6	Operación Parque Rodó	Investigación por robo de un vehículo.	No	Joaquín Castro	Martín Silva
7	Incidente Cordón	Robo ocurrido en un local comercial.	Sí	Sebastián Torres	Lucía Fernández
8	Caso Tres Cruces	Hurto de pertenencias ocurrido en la terminal.	No	Agustín Morales	Diego Rodríguez
9	Operación Carrasco	Robo cometido en una residencia.	No	Santiago Reyes	Sofía Pereira
10	Caso Aguada	Robo de mercadería de un depósito.	No	Leonardo Díaz	Nicolás Gómez

Todos los investigadores asignados a casos tienen rol **DETECTIVE**, como establece el requisito.

Evidencias — 40

Cada caso tiene **4 evidencias** y combina distintos tipos. Los IDs de las evidencias también son autoincrementales.

Caso	Tipo	Fecha	Descripción	Datos específicos
1	Física	10/09/2026	Destornillador encontrado junto a una ventana.	Huellas: Sí
1	Grabación	10/09/2026	Cámara exterior de la vivienda.	Calidad: 4 / Infraganti: Sí
1	Testimonio	11/09/2026	Vecino observó salir a una persona.	Roberto Pérez / ALTO
1	Física	11/09/2026	Guante encontrado en el jardín.	Huellas: No
2	Grabación	02/09/2026	Cámara ubicada frente al comercio.	Calidad: 3 / Infraganti: Sí
2	Física	02/09/2026	Mochila encontrada cerca del comercio.	Huellas: No
2	Testimonio	03/09/2026	Empleado vio al sospechoso.	Ana Gómez / MEDIO

2	Grabación	03/09/2026	Cámara de un comercio vecino.	Calidad: 2 / Infraganti: No
3	Física	20/08/2026	Documentación bancaria incautada.	Huellas: Sí
3	Testimonio	21/08/2026	Declaración de empleado bancario.	Marcos Suárez / ALTO
3	Grabación	21/08/2026	Cámara de cajero automático.	Calidad: 5 / Infraganti: Sí
3	Testimonio	22/08/2026	Declaración de cliente del banco.	Laura Díaz / BAJO
4	Grabación	14/07/2026	Cámara de seguridad del depósito.	Calidad: 4 / Infraganti: Sí
4	Física	14/07/2026	Herramienta utilizada sobre un contenedor.	Huellas: Sí
4	Testimonio	15/07/2026	Guardia observó movimientos sospechosos.	José Martínez / ALTO
4	Física	15/07/2026	Prenda encontrada en el depósito.	Huellas: No
5	Física	05/06/2026	Vaso encontrado dentro de la vivienda.	Huellas: Sí
5	Grabación	05/06/2026	Cámara ubicada en la entrada.	Calidad: 2 / Infraganti: No

5	Testimonio	06/06/2026	Vecina escuchó ruidos de madrugada.	María Rodríguez / MEDIO
5	Grabación	06/06/2026	Cámara de una vivienda cercana.	Calidad: 4 / Infraganti: Sí
6	Física	01/05/2026	Herramienta encontrada junto al vehículo.	Huellas: No
6	Grabación	01/05/2026	Cámara del estacionamiento.	Calidad: 5 / Infraganti: Sí
6	Testimonio	02/05/2026	Testigo vio al sospechoso junto al auto.	Pedro López / ALTO
6	Física	02/05/2026	Botella encontrada dentro del vehículo.	Huellas: Sí
7	Testimonio	10/04/2026	Empleado observó ingresar al sospechoso.	Lucía Alonso / ALTO
7	Grabación	10/04/2026	Video de seguridad del local.	Calidad: 3 / Infraganti: Sí
7	Física	11/04/2026	Bolsa encontrada dentro del comercio.	Huellas: Sí
7	Testimonio	11/04/2026	Persona frente al comercio declaró.	Andrés Costa / BAJO
8	Grabación	12/03/2026	Cámara de la terminal.	Calidad: 4 / Infraganti: Sí

8	Testimonio	12/03/2026	Pasajero presenció el hurto.	Carolina Ramos / MEDIO
8	Física	13/03/2026	Objeto abandonado próximo al lugar.	Huellas: No
8	Grabación	13/03/2026	Cámara de otro sector de la terminal.	Calidad: 2 / Infraganti: No
9	Física	20/02/2026	Herramienta encontrada en el patio.	Huellas: Sí
9	Grabación	20/02/2026	Cámara de seguridad de la residencia.	Calidad: 5 / Infraganti: Sí
9	Testimonio	21/02/2026	Jardinero vio al sospechoso cerca.	Fernando Silva / ALTO
9	Física	21/02/2026	Gorra encontrada junto a una puerta.	Huellas: No
10	Grabación	15/01/2026	Grabación del interior del depósito.	Calidad: 3 / Infraganti: Sí
10	Física	15/01/2026	Caja manipulada durante el robo.	Huellas: Sí
10	Testimonio	16/01/2026	Trabajador vio al sospechoso.	Natalia Fernández / MEDIO
10	Grabación	16/01/2026	Cámara exterior del depósito.	Calidad: 1 / Infraganti: No

Con esto tenés **10 investigadores, 15 sospechosos, 10 casos y 40 evidencias**. También hay variedad suficiente para probar casos abiertos y cerrados, antecedentes sí/no, huellas sí/no, grabaciones con calidad de 1 a 5, infraganti sí/no y testimonios con credibilidad BAJO, MEDIO y ALTO.

Código del proyecto:

Evidencia.cs

```
using System;

namespace Dominio
{
   public abstract class Evidencia : IValidable
   {

      private static int s_ultimoId = 1;


      private int _id;
      private DateTime _fecha;
      private string _descripcion;



      public int Id
      {
         get { return _id; }
      }

      public DateTime Fecha
      {
         get { return _fecha; }
         set { _fecha = value; }
      }

      public string Descripcion
      {
         get { return _descripcion; }
         set { _descripcion = value; }
      }



      public Evidencia(DateTime fecha, string descripcion)
      {
         _id = s_ultimoId++;
```

```
            _fecha = fecha;
            _descripcion = descripcion;

        }

        // Comprueba que la evidencia tenga una descripción
        // y que la fecha de recolección no sea futura.
        public virtual void Validar()
        {
            if (_fecha > DateTime.Today)
            {
                throw new Exception(
                    "La fecha de recolección de la evidencia no puede ser posterior a la fecha actual."
                );
            }

            if (string.IsNullOrEmpty(_descripcion))
            {
                throw new Exception(
                    "La descripción de la evidencia no puede estar vacía."
                );
            }
        }


        public override string ToString()
        {
            return $"Id: {_id} - Descripción: {_descripcion}";
        }
    }
}
```

Credibilidad.cs

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Dominio
{
    public enum Credibilidad
    {
        Bajo,
        Medio,
        Alto
    }
}
```

Grabacion.cs

```
using System;
using System.Collections.Generic;
```

```
using System.Text;

namespace Dominio
{
    public class Grabacion : Evidencia
    {
        private int _calidad;
        private bool _infraganti;

        public int Calidad
        {

            get { return _calidad; }
            set { _calidad = value; }
        }

        public bool Infraganti
        {

            get { return _infraganti; }
            set { _infraganti= value; }
        }

        public Grabacion(
        DateTime fecha,
        string descripcion,
        int calidad,
        bool infraganti
    ) : base (fecha, descripcion)
        {
            _calidad = calidad;
            _infraganti = infraganti;
        }




        // Ejecuta las validaciones de la clase Evidencia
        // y comprueba que la calidad de la grabación esté entre 1 y 5.

        public override void Validar()
        {

            base.Validar();
            if (_calidad < 1 || _calidad > 5 )
            {
                throw new Exception("La calidad de la grabación debe estar entre 1 y 5.");
            }

        }
    }



}
```

Caso.cs

```
namespace Dominio
{
    public class Caso : IValidable
    {
        private static int s_ultimoId = 1;

        private int _id;
        private string _nombre;
        private string _descripcion;
        private bool _cerrado;
        private Sospechoso _sospechosoPrincipal;
        private Investigador _investigador;
        private List<Evidencia> _evidencias;

        public int Id
        {
            get { return _id; }
        }

        public string Nombre
        {
            get { return _nombre; }
            set { _nombre = value; }
        }

        public string Descripcion
        {
            get { return _descripcion; }
            set { _descripcion = value; }
        }

        public bool Cerrado
        {
            get { return _cerrado; }
            set { _cerrado = value; }
        }

        public Sospechoso SospechosoPrincipal
        {
            get { return _sospechosoPrincipal; }
            set { _sospechosoPrincipal = value; }
        }

        public Investigador Investigador
        {
            get { return _investigador; }
            set { _investigador = value; }
        }

        public List<Evidencia> Evidencias
        {
            get { return _evidencias; }
        }
```

```
public Caso(
    string nombre,
    string descripcion,
    bool cerrado,
    Sospechoso sospechosoPrincipal,
    Investigador investigador)
{
    _id = s_ultimoId++;

    _nombre = nombre;
    _descripcion = descripcion;
    _cerrado = cerrado;
    _sospechosoPrincipal = sospechosoPrincipal;
    _investigador = investigador;

    _evidencias = new List<Evidencia>();
}

// Verifica que el caso tenga datos válidos,
// un sospechoso principal y un investigador
// cuyo rol sea DETECTIVE.

public void Validar() {

    if (string.IsNullOrEmpty(_nombre))
    {
        throw new Exception("El nombre no puede estar vacio");
    }

    if (string.IsNullOrEmpty(_descripcion))
    {
        throw new Exception("La descripcion no puede estar vacia");
    }

    if (_sospechosoPrincipal == null)
    {
        throw new Exception("Sospechoso no puede ser nulo");
    }

    if (_investigador == null)
    {
        throw new Exception("El investigador no puede ser nulo");
    }

    if (_investigador.Rol != Rol.DETECTIVE)
    {
        throw new Exception("El investigador del caso debe ser detective");
    }

}

// Registra una evidencia luego de comprobar que no sea nula
// y validar sus datos.

public void AltaEvidencia(Evidencia evidencia)
{
    if (evidencia == null)
```

```
            {
                throw new Exception("La evidencia no puede ser nula");
            }

            evidencia.Validar();

            _evidencias.Add(evidencia);
        }


        public override string ToString()
        {
            return $"Id: {_id} - Nombre: {_nombre}";
        }
    }
}
```

Investigador.cs

```
using System;
using System.Collections.Generic;

namespace Dominio
{
    public class Investigador : IValidable
    {
        private string _email;
        private string _contrasenia;
        private string _nombre;
        private Rol _rol;

        public string Email
        {
            get { return _email; }
            set { _email = value; }
        }

        public string Contrasenia
        {
            get { return _contrasenia; }
            set { _contrasenia = value; }
        }

        public string Nombre
        {
            get { return _nombre; }
            set { _nombre = value; }
        }

        public Rol Rol
        {
            get { return _rol; }
            set { _rol = value; }
        }
```

```
        public Investigador(
            string email,
            string contrasenia,
            string nombre,
            Rol rol)
        {
            _email = email;
            _contrasenia = contrasenia;
            _nombre = nombre;
            _rol = rol;
        }

        // Comprueba que tenga un nombre, mail y contraseña
        public void Validar()
        {
            if (string.IsNullOrEmpty(_nombre))
            {
                throw new Exception("El nombre no puede estar vacio");
            }

            if (string.IsNullOrEmpty(_email))
            {
                throw new Exception("El mail no puede estar vacio");
            }

            if (string.IsNullOrEmpty(_contrasenia))
            {
                throw new Exception("La contraseña no puede estar vacia");
            }
        }

        public override string ToString()
        {
            return $"Nombre: {_nombre} - Email: {_email} - " +
                   $"Rol: {_rol}";
        }


    }
}
```

Sospechoso.cs

```
using System;

namespace Dominio
{
    public class Sospechoso : IValidable
    {

        private string _nombre;
        private string _cedula;
        private bool _antecedentes;
        private DateTime _fechaNacimiento;
```

```
public string Nombre
{
    get { return _nombre; }
    set { _nombre = value; }
}

public string Cedula
{
    get { return _cedula; }
    set { _cedula = value; }
}

public bool Antecedentes
{
    get { return _antecedentes; }
    set { _antecedentes = value; }
}

public DateTime FechaNacimiento
{
    get { return _fechaNacimiento; }
    set { _fechaNacimiento = value; }
}



public Sospechoso(
    string nombre,
    string cedula,
    bool antecedentes,
    DateTime fechaNacimiento)
{
    _nombre = nombre;
    _cedula = cedula;
    _antecedentes = antecedentes;
    _fechaNacimiento = fechaNacimiento;

    Validar();
}


// Valida que el nombre y la cédula del sospechoso no estén vacíos
// y que su fecha de nacimiento no sea posterior a la fecha actual.
public void Validar()
{
    if (string.IsNullOrEmpty(_nombre))
    {
        throw new Exception(
            "El nombre del sospechoso no puede estar vacío."
        );
    }

    if (string.IsNullOrEmpty(_cedula))
    {
        throw new Exception(
            "La cédula del sospechoso no puede estar vacía."
```

```
            );
        }

        if (_fechaNacimiento > DateTime.Today)
        {
            throw new Exception(
                "La fecha de nacimiento no puede ser posterior a la fecha actual."
            );
        }
    }



    public override string ToString()
    {
        return $"Nombre: {_nombre} - Cédula: {_cedula} - " +
               $"Fecha de nacimiento: {_fechaNacimiento.ToShortDateString()} - " +
               $"Antecedentes: {_antecedentes}";
    }
  }
}
```

Fisica.cs

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Dominio
{
   public class Fisica : Evidencia
   {
      private bool _huellas;

      public bool Huellas
      {

         get { return _huellas; }
         set { _huellas = value; }
      }
      public Fisica(
         DateTime fecha,
         string descripcion,
         bool huellas
      ) : base(fecha, descripcion)
      {
         _huellas = huellas;
      }

    }
   }
```

Rol.cs

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Dominio
{
    public enum Rol
    {
        DETECTIVE,
        FISCAL
    }
}
```

Testimonio.cs

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Dominio
{
    public class Testimonio : Evidencia
    {
        private string _nombreTestigo;
        private Credibilidad _credibilidad;

        public string NombreTestigo
        {

            get { return _nombreTestigo; }
            set { _nombreTestigo = value; }
        }


     public Testimonio(
        DateTime fecha,
        string descripcion,
        string nombreTestigo,
        Credibilidad credibilidad
    ) : base(fecha, descripcion)
        {
            _nombreTestigo = nombreTestigo;
            _credibilidad = credibilidad;
        }


        // Ejecuta las validaciones de la clase Evidencia
        // y comprueba que el nombre del testigo no esté vacío.

        public override void Validar()
```

```
        {
            base.Validar();

            if (string.IsNullOrEmpty(_nombreTestigo))
            {
                throw new Exception("El nombre del testigo no puede estar vacío.");
            }
        }
    }
}
```

IValidable.cs

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Dominio
{
    public interface IValidable
    {
        void Validar();
    }
}
```

Program.cs

```
using System;
using System.Collections.Generic;
using Dominio;

namespace Proyecto
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Sistema sistema = new Sistema();

            // Se realiza una sola vez, antes de comenzar el menú.
            sistema.PrecargarDatos();

            string? opcion = "";

            while (opcion != "0")
            {
                Console.WriteLine();
                Console.WriteLine("MENÚ");
                Console.WriteLine("1 - Listar casos con sus evidencias");
                Console.WriteLine("2 - Buscar casos por email del investigador");
                Console.WriteLine("3 - Hacer alta de un sospechoso");
```

```
        Console.WriteLine("4 - Obtener sospechosos con antecedentes");
        Console.WriteLine("0 - Salir");
        Console.Write("Ingrese una opción: ");

        opcion = Console.ReadLine();

        Console.WriteLine();

        switch (opcion)
        {
            case "1":
                ListarCasosConEvidencias(sistema);
                break;

            case "2":
                ListarCasosPorInvestigador(sistema);
                break;

            case "3":
                AltaSospechoso(sistema);
                break;
            case "4":
                ObtenerSospechososConAntecedentes(sistema);
                break;
            case "0":
                Console.WriteLine("Programa finalizado.");
                break;

            default:
                Console.WriteLine("La opción ingresada no es válida.");
                break;
        }
    }
}

static void ObtenerSospechososConAntecedentes(Sistema sistema)
{
    List<Sospechoso> sospechosos = sistema.ObtenerSospechososConAntecedentes();

    foreach (Sospechoso sospechoso in sospechosos)
    {
        Console.WriteLine(sospechoso);
    }
}

static void AltaSospechoso(Sistema sistema)
{
    try
    {
        Console.WriteLine("Ingrese el nombre:");
        string nombre = Console.ReadLine();

        Console.WriteLine("Ingrese la cédula:");
        string cedula = Console.ReadLine();

        Console.WriteLine("Ingrese la fecha de nacimiento (dd/MM/yyyy):");
        DateTime fechaNacimiento = DateTime.Parse(Console.ReadLine());
```

```
        Console.WriteLine("¿Tiene antecedentes? (S/N):");
        string respuesta = Console.ReadLine();

        bool antecedentes = respuesta.ToUpper() == "S";

        Sospechoso sospechoso = new Sospechoso(
            nombre,
            cedula,
            antecedentes,
            fechaNacimiento
        );

        sistema.AltaSospechoso(sospechoso);

        Console.WriteLine("Sospechoso ingresado correctamente.");
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

// Punto a: mostrar todos los casos y sus evidencias.
static void ListarCasosConEvidencias(Sistema sistema)
{
    List<Caso> casos = sistema.ObtenerCasos();

    if (casos.Count == 0)
    {
        Console.WriteLine("No hay casos registrados.");
    }
    else
    {
        foreach (Caso caso in casos)
        {
            Console.WriteLine(
                $"Id: {caso.Id} - Nombre: {caso.Nombre}"
            );

            // Cada evidencia se muestra en un renglón aparte.
            foreach (Evidencia evidencia in caso.Evidencias)
            {
                Console.WriteLine(
                    $"    Id: {evidencia.Id} - " +
                    $"Descripción: {evidencia.Descripcion}"
                );
            }

            Console.WriteLine();
        }
    }
}

// Punto b: mostrar los casos de un investigador por su email.
static void ListarCasosPorInvestigador(Sistema sistema)
{
    Console.Write("Ingrese el email del investigador: ");
    string? email = Console.ReadLine();
```

```
            if (email == null || email.Trim() == "")
            {
                Console.WriteLine("Debe ingresar un email.");
            }
            else
            {
                // Quita espacios al principio y al final.
                email = email.Trim();

                List<Caso> casos = sistema.ObtenerCasosPorEmail(email);

                if (casos.Count == 0)
                {
                    Console.WriteLine(
                        "No se encontraron casos para ese email."
                    );
                }
                else
                {
                    foreach (Caso caso in casos)
                    {
                        Console.WriteLine(
                            $"Id: {caso.Id} - " +
                            $"Nombre: {caso.Nombre} - " +
                            $"Investigador: {caso.Investigador.Nombre}"
                        );
                    }
                }
            }
        }
    }
}
```

Sistema.cs

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Dominio
{
    public class Sistema
    {
        private List<Investigador> _investigadores;
        private List<Caso> _casos;
        private List<Sospechoso> _sospechosos;

        public Sistema()
        {
            _investigadores = new List<Investigador>();
            _casos = new List<Caso>();
            _sospechosos = new List<Sospechoso>();
        }
```

```
// Registra un investigador luego de validar sus datos.
// Verifica que no sea nulo y que no exista otro
// investigador registrado con el mismo email.

public void AltaInvestigador(Investigador investigador)
{
    if (investigador == null)
    {
        throw new Exception("El investigador no puede ser nulo");
    }

    investigador.Validar();

    if (BuscarInvestigador(investigador.Email) != null)
    {
        throw new Exception("Ya existe un investigador con ese email.");
    }

    _investigadores.Add(investigador);
}

// Busca un investigador por su email, sin distinguir
// entre mayúsculas y minúsculas.
// Si no lo encuentra, devuelve null.

private Investigador? BuscarInvestigador(string email)
{
    foreach (Investigador investigador in _investigadores)
    {
        if (investigador.Email.ToLower() == email.ToLower())
        {
            return investigador;
        }
    }

    return null;
}

// Registra un sospechoso luego de validar sus datos
// y comprobar que no exista otro con la misma cédula.

public void AltaSospechoso(Sospechoso sospechoso)
{

    if (sospechoso == null)
    {
        throw new Exception("El sospechoso no puede ser nulo");
    }

    sospechoso.Validar();

    if (BuscarSospechoso(sospechoso.Cedula) != null)
    {
        throw new Exception("Ya existe un sospechoso con esa cédula.");
    }

    _sospechosos.Add(sospechoso);
}
```

```
// Registra un caso luego de validar sus datos
// y comprobar que no sea nulo.

public void AltaCaso(Caso caso)
{

    if (caso == null)
    {
        throw new Exception("El caso no puede ser nulo");
    }

    caso.Validar();

    _casos.Add(caso);
}

// Busca un sospechoso por su cédula.
// Si no existe, devuelve null.

private Sospechoso? BuscarSospechoso(string cedula)
{
    foreach (Sospechoso sospechoso in _sospechosos)
    {
        if (sospechoso.Cedula == cedula)
        {
            return sospechoso;
        }
    }

    return null;
}

// Devuelve la lista de todos los casos registrados en el sistema.
public List<Caso> ObtenerCasos()
{
    return _casos;
}


// Busca los casos asignados a un investigador mediante su email,
// sin distinguir entre mayúsculas y minúsculas.
// Devuelve una lista con los casos encontrados.

public List<Caso> ObtenerCasosPorEmail(string email)
{
    List<Caso> resultado = new List<Caso>();

    foreach (Caso caso in _casos)
    {

        if (caso.Investigador.Email.ToLower() == email.ToLower())
        {
            resultado.Add(caso);
        }
    }

    return resultado;
```

```
}


// Precarga investigadores, sospechosos, casos y evidencias
// ficticias para permitir las pruebas del sistema.

public void PrecargarDatos()
{
    // ========================
    // INVESTIGADORES DETECTIVE
    // ========================

    Investigador investigador1 = new Investigador(
        "martin.silva@fiscalia.uy",
        "Martin123",
        "Martín Silva",
        Rol.DETECTIVE
    );
    AltaInvestigador(investigador1);

    Investigador investigador2 = new Investigador(
        "lucia.fernandez@fiscalia.uy",
        "Lucia456",
        "Lucía Fernández",
        Rol.DETECTIVE
    );
    AltaInvestigador(investigador2);

    Investigador investigador3 = new Investigador(
        "diego.rodriguez@fiscalia.uy",
        "Diego789",
        "Diego Rodríguez",
        Rol.DETECTIVE
    );
    AltaInvestigador(investigador3);

    Investigador investigador4 = new Investigador(
        "sofia.pereira@fiscalia.uy",
        "Sofia321",
        "Sofía Pereira",
        Rol.DETECTIVE
    );
    AltaInvestigador(investigador4);

    Investigador investigador5 = new Investigador(
        "nicolas.gomez@fiscalia.uy",
        "Nico654",
        "Nicolás Gómez",
        Rol.DETECTIVE
    );
    AltaInvestigador(investigador5);


    // =====================
    // INVESTIGADORES FISCAL
    // =====================

    Investigador investigador6 = new Investigador(
```

```
    "valentina.lopez@fiscalia.uy",
    "Vale123",
    "Valentina López",
    Rol.FISCAL
);
AltaInvestigador(investigador6);

Investigador investigador7 = new Investigador(
    "andres.martinez@fiscalia.uy",
    "Andres456",
    "Andrés Martínez",
    Rol.FISCAL
);
AltaInvestigador(investigador7);

Investigador investigador8 = new Investigador(
    "camila.suarez@fiscalia.uy",
    "Cami789",
    "Camila Suárez",
    Rol.FISCAL
);
AltaInvestigador(investigador8);

Investigador investigador9 = new Investigador(
    "federico.costa@fiscalia.uy",
    "Fede321",
    "Federico Costa",
    Rol.FISCAL
);
AltaInvestigador(investigador9);

Investigador investigador10 = new Investigador(
    "paula.ramos@fiscalia.uy",
    "Paula654",
    "Paula Ramos",
    Rol.FISCAL
);
AltaInvestigador(investigador10);


// ============
// SOSPECHOSOS
// ============

Sospechoso sospechoso1 = new Sospechoso(
    "Carlos Méndez",
    "4.521.630-1",
    true,
    new DateTime(1985, 5, 14)
);
AltaSospechoso(sospechoso1);

Sospechoso sospechoso2 = new Sospechoso(
    "Bruno Acosta",
    "5.184.920-3",
    false,
    new DateTime(1992, 9, 22)
);
```

```
AltaSospechoso(sospechoso2);

Sospechoso sospechoso3 = new Sospechoso(
    "Mateo Cabrera",
    "4.792.315-6",
    true,
    new DateTime(1988, 2, 3)
);
AltaSospechoso(sospechoso3);

Sospechoso sospechoso4 = new Sospechoso(
    "Rodrigo Núñez",
    "5.321.874-2",
    false,
    new DateTime(1995, 11, 17)
);
AltaSospechoso(sospechoso4);

Sospechoso sospechoso5 = new Sospechoso(
    "Gonzalo Viera",
    "4.638.291-5",
    true,
    new DateTime(1983, 7, 28)
);
AltaSospechoso(sospechoso5);

Sospechoso sospechoso6 = new Sospechoso(
    "Joaquín Castro",
    "5.472.103-8",
    false,
    new DateTime(1998, 1, 11)
);
AltaSospechoso(sospechoso6);

Sospechoso sospechoso7 = new Sospechoso(
    "Sebastián Torres",
    "4.819.562-4",
    true,
    new DateTime(1990, 6, 6)
);
AltaSospechoso(sospechoso7);

Sospechoso sospechoso8 = new Sospechoso(
    "Agustín Morales",
    "5.203.741-9",
    false,
    new DateTime(1994, 3, 19)
);
AltaSospechoso(sospechoso8);

Sospechoso sospechoso9 = new Sospechoso(
    "Santiago Reyes",
    "4.376.918-7",
    true,
    new DateTime(1981, 12, 25)
);
AltaSospechoso(sospechoso9);
```

```
Sospechoso sospechoso10 = new Sospechoso(
    "Emiliano Sosa",
    "5.619.284-1",
    false,
    new DateTime(1997, 8, 8)
);
AltaSospechoso(sospechoso10);

Sospechoso sospechoso11 = new Sospechoso(
    "Leonardo Díaz",
    "4.927.350-6",
    true,
    new DateTime(1987, 4, 30)
);
AltaSospechoso(sospechoso11);

Sospechoso sospechoso12 = new Sospechoso(
    "Franco Olivera",
    "5.738.421-3",
    false,
    new DateTime(2000, 10, 15)
);
AltaSospechoso(sospechoso12);

Sospechoso sospechoso13 = new Sospechoso(
    "Mauricio Lima",
    "4.583.127-9",
    true,
    new DateTime(1984, 1, 9)
);
AltaSospechoso(sospechoso13);

Sospechoso sospechoso14 = new Sospechoso(
    "Facundo Pereira",
    "5.346.892-5",
    false,
    new DateTime(1996, 6, 21)
);
AltaSospechoso(sospechoso14);

Sospechoso sospechoso15 = new Sospechoso(
    "Pablo Ríos",
    "4.715.263-8",
    true,
    new DateTime(1989, 3, 12)
);
AltaSospechoso(sospechoso15);


// ======
// CASOS
// ======

Caso caso1 = new Caso(
    "Robo en Pocitos",
    "Robo ocurrido durante la madrugada en una vivienda de Pocitos.",
    false,
    sospechoso1,
```

```
    investigador1
);
AltaCaso(caso1);

Caso caso2 = new Caso(
    "Hurto del Centro",
    "Hurto de equipos electrónicos en un comercio del Centro.",
    false,
    sospechoso2,
    investigador2
);
AltaCaso(caso2);

Caso caso3 = new Caso(
    "Fraude Bancario",
    "Investigación por movimientos bancarios fraudulentos.",
    false,
    sospechoso3,
    investigador3
);
AltaCaso(caso3);

Caso caso4 = new Caso(
    "Robo del Puerto",
    "Robo de mercadería almacenada en la zona portuaria.",
    true,
    sospechoso4,
    investigador4
);
AltaCaso(caso4);

Caso caso5 = new Caso(
    "Caso Malvín",
    "Investigación de un ingreso ilegal a una vivienda en Malvín.",
    false,
    sospechoso5,
    investigador5
);
AltaCaso(caso5);

Caso caso6 = new Caso(
    "Operación Parque Rodó",
    "Investigación por robo de un vehículo próximo al Parque Rodó.",
    false,
    sospechoso6,
    investigador1
);
AltaCaso(caso6);

Caso caso7 = new Caso(
    "Incidente Cordón",
    "Robo en un local comercial durante el horario nocturno.",
    true,
    sospechoso7,
    investigador2
);
AltaCaso(caso7);
```

```
Caso caso8 = new Caso(
    "Caso Tres Cruces",
    "Investigación por hurto de pertenencias en la terminal.",
    false,
    sospechoso8,
    investigador3
);
AltaCaso(caso8);

Caso caso9 = new Caso(
    "Operación Carrasco",
    "Investigación de un robo cometido en una residencia.",
    false,
    sospechoso9,
    investigador4
);
AltaCaso(caso9);

Caso caso10 = new Caso(
    "Caso Aguada",
    "Investigación por robo de mercadería de un depósito.",
    false,
    sospechoso11,
    investigador5
);
AltaCaso(caso10);


// ====================
// EVIDENCIAS - CASO 1
// ====================

Fisica evidencia1 = new Fisica(
    new DateTime(2026, 9, 10),
    "Destornillador encontrado junto a una ventana forzada.",
    true
);
caso1.AltaEvidencia(evidencia1);

Grabacion evidencia2 = new Grabacion(
    new DateTime(2026, 9, 10),
    "Video de una cámara exterior de la vivienda.",
    4,
    true
);
caso1.AltaEvidencia(evidencia2);

Testimonio evidencia3 = new Testimonio(
    new DateTime(2026, 9, 11),
    "Vecino observó a una persona salir de la vivienda.",
    "Roberto Pérez",
    Credibilidad.Alto
);
caso1.AltaEvidencia(evidencia3);

Fisica evidencia4 = new Fisica(
    new DateTime(2026, 9, 11),
    "Guante encontrado en el jardín de la propiedad.",
```

```
    false
);
caso1.AltaEvidencia(evidencia4);


// ====================
// EVIDENCIAS - CASO 2
// ====================

Grabacion evidencia5 = new Grabacion(
    new DateTime(2026, 9, 2),
    "Cámara ubicada frente al comercio.",
    3,
    true
);
caso2.AltaEvidencia(evidencia5);

Fisica evidencia6 = new Fisica(
    new DateTime(2026, 9, 2),
    "Mochila encontrada cerca del comercio.",
    false
);
caso2.AltaEvidencia(evidencia6);

Testimonio evidencia7 = new Testimonio(
    new DateTime(2026, 9, 3),
    "Empleado declaró haber visto al sospechoso.",
    "Ana Gómez",
    Credibilidad.Medio
);
caso2.AltaEvidencia(evidencia7);

Grabacion evidencia8 = new Grabacion(
    new DateTime(2026, 9, 3),
    "Video grabado desde un comercio vecino.",
    2,
    false
);
caso2.AltaEvidencia(evidencia8);


// ====================
// EVIDENCIAS - CASO 3
// ====================

Fisica evidencia9 = new Fisica(
    new DateTime(2026, 8, 20),
    "Documentación bancaria encontrada durante un allanamiento.",
    true
);
caso3.AltaEvidencia(evidencia9);

Testimonio evidencia10 = new Testimonio(
    new DateTime(2026, 8, 21),
    "Declaración de un empleado bancario.",
    "Marcos Suárez",
    Credibilidad.Alto
);
```

```
caso3.AltaEvidencia(evidencia10);

Grabacion evidencia11 = new Grabacion(
    new DateTime(2026, 8, 21),
    "Cámara del cajero automático investigado.",
    5,
    true
);
caso3.AltaEvidencia(evidencia11);

Testimonio evidencia12 = new Testimonio(
    new DateTime(2026, 8, 22),
    "Declaración de un cliente del banco.",
    "Laura Díaz",
    Credibilidad.Bajo
);
caso3.AltaEvidencia(evidencia12);


// ====================
// EVIDENCIAS - CASO 4
// ====================

Grabacion evidencia13 = new Grabacion(
    new DateTime(2026, 7, 14),
    "Grabación de seguridad del depósito.",
    4,
    true
);
caso4.AltaEvidencia(evidencia13);

Fisica evidencia14 = new Fisica(
    new DateTime(2026, 7, 14),
    "Herramienta utilizada para abrir un contenedor.",
    true
);
caso4.AltaEvidencia(evidencia14);

Testimonio evidencia15 = new Testimonio(
    new DateTime(2026, 7, 15),
    "Guardia observó movimientos sospechosos.",
    "José Martínez",
    Credibilidad.Alto
);
caso4.AltaEvidencia(evidencia15);

Fisica evidencia16 = new Fisica(
    new DateTime(2026, 7, 15),
    "Prenda encontrada dentro del depósito.",
    false
);
caso4.AltaEvidencia(evidencia16);


// ====================
// EVIDENCIAS - CASO 5
// ====================
```

```
Fisica evidencia17 = new Fisica(
    new DateTime(2026, 6, 5),
    "Vaso encontrado dentro de la vivienda.",
    true
);
caso5.AltaEvidencia(evidencia17);

Grabacion evidencia18 = new Grabacion(
    new DateTime(2026, 6, 5),
    "Cámara ubicada en la entrada.",
    2,
    false
);
caso5.AltaEvidencia(evidencia18);

Testimonio evidencia19 = new Testimonio(
    new DateTime(2026, 6, 6),
    "Vecina escuchó ruidos durante la madrugada.",
    "María Rodríguez",
    Credibilidad.Medio
);
caso5.AltaEvidencia(evidencia19);

Grabacion evidencia20 = new Grabacion(
    new DateTime(2026, 6, 6),
    "Cámara de una vivienda cercana.",
    4,
    true
);
caso5.AltaEvidencia(evidencia20);


// ====================
// EVIDENCIAS - CASO 6
// ====================

Fisica evidencia21 = new Fisica(
    new DateTime(2026, 5, 1),
    "Herramienta encontrada junto al vehículo.",
    false
);
caso6.AltaEvidencia(evidencia21);

Grabacion evidencia22 = new Grabacion(
    new DateTime(2026, 5, 1),
    "Cámara del estacionamiento.",
    5,
    true
);
caso6.AltaEvidencia(evidencia22);

Testimonio evidencia23 = new Testimonio(
    new DateTime(2026, 5, 2),
    "Testigo vio al sospechoso junto al automóvil.",
    "Pedro López",
    Credibilidad.Alto
);
caso6.AltaEvidencia(evidencia23);
```

```
Fisica evidencia24 = new Fisica(
    new DateTime(2026, 5, 2),
    "Botella encontrada dentro del vehículo.",
    true
);
caso6.AltaEvidencia(evidencia24);


// ====================
// EVIDENCIAS - CASO 7
// ====================

Testimonio evidencia25 = new Testimonio(
    new DateTime(2026, 4, 10),
    "Empleado observó el ingreso del sospechoso.",
    "Lucía Alonso",
    Credibilidad.Alto
);
caso7.AltaEvidencia(evidencia25);

Grabacion evidencia26 = new Grabacion(
    new DateTime(2026, 4, 10),
    "Video de seguridad del local.",
    3,
    true
);
caso7.AltaEvidencia(evidencia26);

Fisica evidencia27 = new Fisica(
    new DateTime(2026, 4, 11),
    "Bolsa encontrada dentro del comercio.",
    true
);
caso7.AltaEvidencia(evidencia27);

Testimonio evidencia28 = new Testimonio(
    new DateTime(2026, 4, 11),
    "Persona ubicada frente al comercio declaró lo sucedido.",
    "Andrés Costa",
    Credibilidad.Bajo
);
caso7.AltaEvidencia(evidencia28);


// ====================
// EVIDENCIAS - CASO 8
// ====================

Grabacion evidencia29 = new Grabacion(
    new DateTime(2026, 3, 12),
    "Grabación de una cámara de la terminal.",
    4,
    true
);
caso8.AltaEvidencia(evidencia29);

Testimonio evidencia30 = new Testimonio(
```

```
    new DateTime(2026, 3, 12),
    "Pasajero vio el momento del hurto.",
    "Carolina Ramos",
    Credibilidad.Medio
);
caso8.AltaEvidencia(evidencia30);

Fisica evidencia31 = new Fisica(
    new DateTime(2026, 3, 13),
    "Objeto abandonado próximo al lugar.",
    false
);
caso8.AltaEvidencia(evidencia31);

Grabacion evidencia32 = new Grabacion(
    new DateTime(2026, 3, 13),
    "Grabación desde otro sector de la terminal.",
    2,
    false
);
caso8.AltaEvidencia(evidencia32);


// ====================
// EVIDENCIAS - CASO 9
// ====================

Fisica evidencia33 = new Fisica(
    new DateTime(2026, 2, 20),
    "Herramienta encontrada en el patio.",
    true
);
caso9.AltaEvidencia(evidencia33);

Grabacion evidencia34 = new Grabacion(
    new DateTime(2026, 2, 20),
    "Cámara de seguridad de la residencia.",
    5,
    true
);
caso9.AltaEvidencia(evidencia34);

Testimonio evidencia35 = new Testimonio(
    new DateTime(2026, 2, 21),
    "Jardinero observó al sospechoso cerca de la vivienda.",
    "Fernando Silva",
    Credibilidad.Alto
);
caso9.AltaEvidencia(evidencia35);

Fisica evidencia36 = new Fisica(
    new DateTime(2026, 2, 21),
    "Gorra encontrada junto a una puerta lateral.",
    false
);
caso9.AltaEvidencia(evidencia36);
```

```
        // ======================
        // EVIDENCIAS - CASO 10
        // ======================

        Grabacion evidencia37 = new Grabacion(
            new DateTime(2026, 1, 15),
            "Grabación del interior del depósito.",
            3,
            true
        );
        caso10.AltaEvidencia(evidencia37);

        Fisica evidencia38 = new Fisica(
            new DateTime(2026, 1, 15),
            "Caja manipulada durante el robo.",
            true
        );
        caso10.AltaEvidencia(evidencia38);

        Testimonio evidencia39 = new Testimonio(
            new DateTime(2026, 1, 16),
            "Trabajador declaró haber visto al sospechoso.",
            "Natalia Fernández",
            Credibilidad.Medio
        );
        caso10.AltaEvidencia(evidencia39);

        Grabacion evidencia40 = new Grabacion(
            new DateTime(2026, 1, 16),
            "Grabación de una cámara exterior.",
            1,
            false
        );
        caso10.AltaEvidencia(evidencia40);
    }

    public List<Sospechoso> ObtenerSospechososConAntecedentes()
    {
        List<Sospechoso> sospechososConAntecedentes = new List<Sospechoso>();

        foreach (Sospechoso sospechoso in _sospechosos)
        {
            if (sospechoso.Antecedentes)
            {
                sospechososConAntecedentes.Add(sospechoso);
            }
        }

        return sospechososConAntecedentes;
    }

}


}
```